

documentar programas 20
 erro de compilação 25
 erro de sintaxe 25
 erro no tempo de compilação 25
 espaço em branco 34
 executável 22
 false 32
 fluxo de controle 35
 formato em linha 29
 função 21
 identificador 24
 instrução 21
 instrução de atribuição 26
 instrução `i f` 32
 inteiros 24
 literal 21
 mensagem 21
 modelo de ação/decisão 22
 não destrutivo 28
 nova linha (`\n`) 21
 operador de endereço (&) 25
 operadores aritméticos 28
 operadores de igualdade 32
 operadores relacionais 32
 operandos 26
 palavras-chave 35
 parênteses aninhados 29
 parênteses aninhados (embutidos) 29
 parênteses redundantes 31
 pré-processor C 21
 programação estruturada 20
 prompt 25
 regras de precedência de operador 29
 sair de uma função 22
 sensível a maiúsculas/minúsculas 24
 sequência de escape 21
 sinal de porcentagem (%) 28
 string de caracteres 21
 string de controle de formato 25
 tecla *Enter* 26
 terminador de instrução (;) 21
 tipo 28
 true 32
 valor 28
 variáveis 24

■ Exercícios de autorrevisão

2.1 Preencha os espaços nas seguintes frases:

- a) Todo programa em C inicia sua execução pela função _____.
- b) O(a) _____ inicia o corpo de cada função, e o(a) _____ encerra o corpo de cada função.
- c) Toda instrução termina com um(a) _____.
- d) A função _____ da biblioteca-padrão exibe informações na tela.
- e) A sequência de escape `\n` representa o caractere de _____, que faz com que o cursor se posicione no início da próxima linha na tela.
- f) A função _____ da biblioteca-padrão é usada para obter dados do teclado.
- g) O especificador de conversão _____ é usado em uma string de controle de formato de `scanf` para indicar que um inteiro será digitado, e em uma string de controle de formato de `printf` para indicar que um inteiro será exibido.
- h) Sempre que um novo valor é colocado em uma posição da memória, ele anula o valor que ocupava essa mesma posição anteriormente. Esse processo é chamado de _____.

- i) Quando um valor é lido de uma posição da memória, o valor nessa posição é preservado; esse processo é chamado de _____.
- j) A instrução _____ é usada na tomada de decisões.

2.2 Indique se cada uma das sentenças a seguir é *verdadeira* ou *falsa*. Explique sua resposta no caso de alternativas *falsas*.

- a) A função `printf` sempre começa a imprimir no início de uma nova linha.
- b) Comentários fazem com que o computador imprima na tela o texto delimitado por `/*` e `*/` quando o programa é executado.
- c) A sequência de escape `\n`, quando usada em uma string de controle de formato de `printf`, faz com que o cursor se posicione no início da próxima linha na tela.
- d) Todas as variáveis precisam ser declaradas antes de serem usadas.
- e) Todas as variáveis precisam receber um tipo ao serem declaradas.
- f) C considera as variáveis `número` e `NúMeRo` idênticas.

- g) Declarações podem aparecer em qualquer parte do corpo de uma função.
- h) Todos os argumentos após a string de controle de formato em uma função `printf` precisam ser precedidos por `(&)`.
- i) O operador módulo (%) só pode ser usado com operandos inteiros.
- j) Os operadores aritméticos `*`, `/`, `%`, `+` e `-` têm o mesmo nível de precedência.
- k) Os nomes das variáveis a seguir são idênticos em todos os sistemas C padrão.

`esteéumnúmerosuperhiperlongo1234567`
`esteéumnúmerosuperhiperlongo1234568`

- l) Um programa que exibe três linhas de saída precisa conter três instruções `printf`.

2.3 Escreva uma única instrução C para executar cada uma das seguintes alternativas:

- a) Declarar as variáveis `c`, `estaVariável`, `q76354` e número para que tenham o tipo `int`.
- b) Pedir que o usuário digite um inteiro. Terminar a mensagem com um sinal de dois pontos (:) seguido por um espaço, e deixar o cursor posicionado após o espaço.
- c) Ler um inteiro do teclado e armazenar o valor digitado na variável inteira `a`.
- d) Se o número não for igual a 7, exibir “A variável número não é igual a 7”.
- e) Imprimir a mensagem “Este é um programa em C.” em uma linha.
- f) Imprimir a mensagem “Este é um programa em C.” em duas linhas, de modo que a primeira linha termine em C.

- g) Imprimir a mensagem “Este é um programa em C.” com cada palavra em uma linha separada.
- h) Imprimir a mensagem “Este é um programa em C.” com as palavras separadas por tabulações.

2.4 Escreva uma instrução (ou comentário) para realizar cada um dos seguintes:

- a) Indicar que um programa calculará o produto de três inteiros.
- b) Declarar as variáveis `x`, `y`, `z` e `resultado` para que tenham o tipo `int`.
- c) Pedir ao usuário que digite três inteiros.
- d) Ler três inteiros do teclado e armazená-los nas variáveis `x`, `y` e `z`.
- e) Calcular o produto dos três inteiros contidos nas variáveis `x`, `y` e `z`, e atribuir o resultado à variável `resultado`.
- f) Exibir “O produto é” seguido pelo valor da variável inteira `resultado`.

2.5 Usando as instruções que você escreveu no Exercício 2.4, escreva um programa completo que calcule o produto de três inteiros.

2.6 Identifique e corrija os erros em cada uma das seguintes instruções:

- a) `printf( “O valor é %d\n”, &número );`
- b) `scanf( “%d%d”, &número1, número2 );`
- c) `if ( c < 7 );{`
`printf( “C é menor que 7\n” );`
`}`
- d) `if ( c ==> 7 ) {`
`printf( “C é igual ou menor que`
`7\n” );`
`}`

Respostas dos exercícios de autorrevisão

2.1 a) `main`. b) chave à esquerda (`{`), chave à direita (`}`). c) ponto e vírgula. d) `printf`. e) nova linha. f) `scanf`. g) `%d`. h) destrutivo. i) não destrutivo. j) `if`.

2.2 a) Falso. A função `printf` sempre começa a imprimir onde o cursor estiver posicionado, e isso pode ser em qualquer lugar de uma linha na tela.
 b) Falso. Os comentários não causam qualquer ação quando o programa é executado. Eles são usados para documentar programas e melhorar sua legibilidade.
 c) Verdadeiro.
 d) Verdadeiro.
 e) Verdadeiro.

f) Falso. C diferencia maiúsculas de minúsculas, de modo que essas variáveis são diferentes.

g) Falso. As definições precisam aparecer após a chave à esquerda do corpo de uma função e antes de quaisquer instruções executáveis.

h) Falso. Os argumentos em uma função `printf` normalmente não devem ser precedidos por um `(&)`. Os argumentos após a string de controle de formato em uma função `scanf` normalmente devem ser precedidos por um `(&)`. Discutiremos as exceções a essas regras nos capítulos 6 e 7.

i) Verdadeiro.

- j) Falso. Os operadores *, / e % estão no mesmo nível de precedência, e os operadores + e - estão em um nível de precedência mais baixo.
- k) Falso. Alguns sistemas podem distinguir os identificadores com mais de 31 caracteres.
- l) Falso. Uma instrução printf com várias sequências de escape \n pode exibir várias linhas.

- 2.3** a) `int c`, estaVariável, q76354, número;
 b) `printf( "Digite um inteiro:" );`
 c) `scanf( "%d", &a );`
 d) `if ( número != 7 )`

```
{
    printf( "A variável número não é
    igual a 7.\n" );
}
```

- e) `printf( "Este é um programa em C.\n" );`
- f) `printf( "Este é um programa em\nC.\n" );`
- g) `printf( "Este\né\num\nprograma\nem\nC.\n" );`
- h) `printf( "Este\até\tum\tprograma\tem\tC.\n" );`

- 2.4** a) `/* Calcula o produto de três inteiros */`
 b) `int x, y, z, resultado;`
 c) `printf( "Digite três inteiros: " );`
 d) `scanf( "%d%d%d", &x, &y, &z );`
 e) `result = x * y * z;`
 f) `printf( "O produto é %d\n", resultado );`

- 2.5** Veja a seguir.

```
1  /* Calcula o produto de três inteiros
   */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      int x, y, z, resultado; /* declara
   variáveis */
7
8      printf( "Digite três inteiros: " );
   /* prompt */
9      scanf( "%d%d%d", &x, &y, &z ); /*
   lê três inteiros */
10     result = x * y * z; /* multiplica
   os valores */
11     printf( "O produto é %d\n",
   result ); /* exibe o resultado */
12     return 0;
13 } /* fim da função main */
```

- 2.6** a) Erro: `&número`. Correção: elimine o (&). Discutiremos as exceções mais à frente.
 b) Erro: `número2` não tem um (&). Correção: `número2` deveria ser `&número2`. Adiante no texto, discutiremos as exceções.
 c) Erro: o ponto e vírgula após o parêntese direito da condição na instrução `if`. Correção: remova o ponto e vírgula após o parêntese direito. [Nota: o resultado desse erro é que a instrução `printf` será executada, sendo ou não verdadeira a condição na instrução `if`. O ponto e vírgula após o parêntese direito é considerado uma instrução vazia, uma instrução que não faz nada.]
 d) Erro: o operador relacional `=>` deve ser mudado para `>=` (maior ou igual a).

Exercícios

- 2.7** Identifique e corrija os erros cometidos em cada uma das instruções. (Nota: pode haver mais de um erro por instrução.)

- a) `scanf( "d", valor );`
- b) `printf( "O produto de %d e %d é %d"\n, x, y );`
- c) `primeiroNúmero + segundoNúmero = somaDosNúmeros`
- d) `if ( número => maior )`
`maior == número;`
- e) `/* Programa para determinar o maior dentre três inteiros */`
- f) `Scanf( "%d", umInteiro );`
- g) `printf( "Módulo de %d dividido por %d é\n", x, y, x % y );`
- h) `if ( x = y );`
`printf( %d é igual a %d\n", x, y );`

- i) `print( "A soma é %d\n," x + y );`
- j) `Printf( "O valor que você digitou é: %d\n, &valor );`

- 2.8** Preencha os espaços nas seguintes sentenças:

- a) _____ são usados para documentar um programa e melhorar sua legibilidade.
- b) A função usada para exibir informações na tela é _____.
- c) Uma instrução C responsável pela tomada de decisão é _____.
- d) Os cálculos normalmente são realizados por instruções _____.
- e) A função _____ lê valores do teclado.

- 2.9** Escreva uma única instrução em C, ou linha única, que cumpra os comentários a seguir:

- a) Exiba a mensagem 'Digite dois números.'
- b) Atribua o produto das variáveis `b` e `c` à variável `a`.

- c) Indique um programa que realize um cálculo de folha de pagamento (ou seja, use um texto que ajude a documentar um programa).
- d) Informe três valores inteiros usando o teclado e coloque esses valores nas variáveis inteiras a, b e c.
- 2.10** Informe quais das seguintes sentenças são *verdadeiras* e quais são *falsas*. Explique sua resposta no caso de sentenças falsas.
- a) Os operadores em C são avaliados da esquerda para a direita.
- b) Os seguintes nomes de variáveis são válidos: `_sub_barra_`, `m928134`, `t5`, `j7`, `suas_vendas`, `total_sua_conta`, `a`, `b`, `c`, `z`, `z2`.
- c) A instrução `printf("a = 5;");` é um exemplo típico de uma instrução de atribuição.
- d) Uma expressão aritmética válida sem parênteses é avaliada da esquerda para a direita.
- e) Os seguintes nomes de variáveis são inválidos: `3g`, `87`, `67h2`, `h22`, `2h`.
- 2.11** Preencha os espaços de cada sentença:
- a) Que operações aritméticas estão no mesmo nível de precedência da multiplicação? _____
- b) Quando os parênteses são aninhados, qual conjunto de parênteses é avaliado em primeiro lugar em uma expressão aritmética? _____.
- c) Uma posição na memória do computador que pode conter diferentes valores em vários momentos durante a execução de um programa é chamada de _____.
- 2.12** O que é exibido quando cada uma das seguintes instruções é executada? Se nada for exibido, então responda 'Nada'. Considere que `x = 2` e `y = 3`.
- a) `printf( "%d", x );`
- b) `printf( "%d", x + x );`
- c) `printf( "x=" );`
- d) `printf( "x=%d", x );`
- e) `printf( "%d = %d", x + y, y + x );`
- f) `z = x + y;`
- g) `scanf( "%d%d", &x, &y );`
- h) `/* printf( "x + y = %d", x + y ); */`
- i) `printf( "\n" );`
- 2.13** Quais das seguintes instruções em C contêm variáveis cujos valores são substituídos?
- a) `scanf( "%d%d%d%d", &b, &c, &d, &e, &f );`
- b) `p = i + j + k + 7;`
- c) `printf( "Valores são substituídos" );`
- d) `printf( "a = 5" );`
- 2.14** Dada a equação $y = ax^3 + 7$, quais das seguintes alternativas são instruções em C corretas para essa equação (se houver alguma)?
- a) `y = a * x * x * x + 7;`
- b) `y = a * x * x * ( x + 7 );`
- c) `y = ( a * x ) * x * ( x + 7 );`
- d) `y = ( a * x ) * x * x + 7;`
- e) `y = a * ( x * x * x ) + 7;`
- f) `y = a * x * ( x * x + 7 );`
- 2.15** Indique a ordem de avaliação dos operadores em cada uma das instruções em C a seguir, e mostre o valor de x após a execução de cada instrução.
- a) `x = 7 + 3 * 6 / 2 - 1;`
- b) `x = 2 % 2 + 2 * 2 - 2 / 2;`
- c) `x = ( 3 * 9 * ( 3 + ( 9 * 3 / ( 3 ) ) ) );`
- 2.16** **Aritmética.** Escreva um programa que peça ao usuário que digite dois números, obtenha esses números e imprima a soma, o produto, a diferença, o quociente e o módulo (resto da divisão).
- 2.17** **Imprimindo valores com printf.** Escreva um programa que imprima os números de 1 a 4 na mesma linha. Escreva o programa utilizando os métodos a seguir.
- a) Uma instrução `printf` sem especificadores de conversão.
- b) Uma instrução `printf` com quatro especificadores de conversão.
- c) Quatro instruções `printf`.
- 2.18** **Comparando inteiros.** Escreva um programa que peça ao usuário que digite dois inteiros, obtenha os números e depois imprima o maior número seguido das palavras 'é maior'. Se os números forem iguais, imprima a mensagem "Esses números são iguais". Use apenas a forma de seleção única da instrução `if` que você aprendeu neste capítulo.
- 2.19** **Aritmética, maior e menor valor.** Escreva um programa que leia três inteiros diferentes do teclado, depois apresente a soma, a média, o produto, o menor e o maior desses números. Use apenas a forma de seleção única da instrução `if` que você aprendeu neste capítulo. O diálogo na tela deverá aparecer da seguinte forma:
- Digite três inteiros diferentes: 13 27 14

A soma é 54

A média é 18

O produto é 4914

O menor é 13

O maior é 27
- 2.20** **Diâmetro, circunferência e área de um círculo.** Escreva um programa que leia o raio de um círculo e informe o diâmetro, a circunferência e a área do círculo. Utilize o valor constante 3,14159 para π . Realize cada um desses cálculos dentro das instruções `printf` e use

o especificador de conversão %f. [Nota: neste capítulo, discutimos apenas constantes e variáveis inteiras. No Capítulo 3, discutiremos os números em ponto flutuante, ou seja, valores que podem ter pontos decimais.]

2.21 Formas com asteriscos. Escreva um programa que imprima as seguintes formas com asteriscos.

```
*****      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*****      *      *
```

2.22 O que o código a seguir imprime?

```
printf( " *\n**\n***\n****\n*****\n" );
```

2.23 Maiores e menores inteiros. Escreva um programa que leia cinco inteiros e depois determine e imprima o maior e o menor inteiro no grupo. Use apenas as técnicas de programação que você aprendeu neste capítulo.

2.24 Par ou ímpar. Escreva um programa que leia um inteiro, determine e imprima se ele é par ou ímpar. [Dica: use o operador módulo. Um número par é um múltiplo de dois. Qualquer múltiplo de dois gera resto zero quando dividido por 2.]

2.25 Imprima as suas iniciais em letras em bloco no sentido vertical da página. Construa cada letra em bloco a partir da letra que ele representa, como mostramos a seguir.

```
PPPPPPPP
  P  P
  P  P
  P  P
  PP

  JJ
  J
  J
  J
  JJJJJJ

DDDDDDDD
D      D
D      D
D      D
DDDDD
```

2.26 Múltiplos. Escreva um programa que leia dois inteiros, determine e imprima se o primeiro for um múltiplo do segundo. [Dica: use o operador módulo.]

2.27 Padrão de asteriscos alternados. Apresente o seguinte padrão de asteriscos alternados com oito instruções printf, e depois apresente o mesmo padrão com o mínimo de instruções printf possível.

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

2.28 Faça a distinção entre os termos erro fatal e erro não fatal. Por que seria preferível experimentar um erro fatal em vez de um erro não fatal?

2.29 Valor inteiro de um caractere. Vamos dar um passo adiante. Neste capítulo, você aprendeu sobre inteiros e o tipo int. C também pode representar letras maiúsculas, letras minúsculas e uma grande variedade de símbolos especiais. C usa inteiros pequenos internamente para representar diferentes caracteres. O conjunto de caracteres que um computador utiliza, juntamente com as representações de inteiros correspondentes a esses caracteres, é chamado de conjunto de caracteres desse computador. Você pode imprimir o equivalente da letra A maiúscula, por exemplo, executando a instrução

```
printf( "%d", 'A' );
```

Escreva um programa em C que imprima os equivalentes inteiros de algumas letras maiúsculas, letras minúsculas, dígitos e símbolos especiais. No mínimo, determine os equivalentes inteiros de A B C a b c 0 1 2 \$ * + / e o caractere de espaço em branco.

2.30 Separando dígitos em um inteiro. Escreva um programa que leia um número de cinco dígitos, separe o número em dígitos individuais e imprima os dígitos separados um do outro por três espaços cada um. [Dica: use combinações da divisão inteira e da operação módulo.] Por exemplo, se o usuário digitar 42139, o programa deverá exibir

```
4 2 1 3 9
```

2.31 Tabela de quadrados e cubos. Usando apenas as técnicas que você aprendeu neste capítulo, escreva um programa que calcule os quadrados e os cubos dos números 0 a 10, e use tabulações para imprimir a seguinte tabela de valores:

número	quadrado	cubo
0	0	0
1	1	1
2	4	8
3	9	27
4	16	64
5	25	125
6	36	216
7	49	343
8	64	512
9	81	729
10	100	1000

■ Fazendo a diferença

2.32 Calculadora de Índice de Massa Corporal. Apresentamos a calculadora do índice de massa corporal (IMC) no Exercício 1.11. A fórmula para calcular o IMC é

$$IMC = \frac{\text{pesoEmQuilos}}{\text{alturaEmMetros} \times \text{alturaEmMetros}}$$

Crie uma aplicação para a calculadora de IMC que leia o peso do usuário em quilogramas e a altura em metros, e que depois calcule e apresente o seu índice de massa corporal. Além disso, a aplicação deverá exibir as seguintes informações do Ministério da Saúde para que o usuário possa avaliar seu IMC:

VALORES DE IMC	
Abaixo do peso:	menor que 18,5
Normal:	entre 18,5 e 24,9
Acima do peso:	entre 25 e 29,9
Obeso:	30 ou mais

[Nota: neste capítulo, você aprendeu a usar o tipo `int` para representar números inteiros. Os cálculos de IMC, quando feitos com valores `int`, produzirão resultados

em números inteiros. No Capítulo 4, você aprenderá a usar o tipo `double` para representar números fracionários. Quando os cálculos de IMC são realizados com `doubles`, eles produzem números com pontos decimais; estes são os chamados números de 'ponto flutuante'.]

2.33 Calculadora de economias com o transporte solidário. Pesquise diversos websites sobre transporte solidário com carros de passeio. Crie uma aplicação que calcule a sua despesa diária com o automóvel, para que você possa estimar quanto dinheiro poderia economizar com o transporte solidário, que também tem outras vantagens, como reduzir as emissões de carbono e os congestionamentos. A aplicação deverá solicitar as seguintes informações, e exibir os custos com o trajeto diário ao trabalho:

- a) Total de quilômetros dirigidos por dia.
- b) Custo por litro de combustível.
- c) Média de quilômetros por litro.
- d) Preço de estacionamento por dia.
- e) Gastos diários com pedágios.